



Technische Universität Darmstadt
Referat Zulassung International
Karolinenplatz 5
64289 Darmstadt



Familienname	Riaz
Vorname	Hammad
Geburtsdatum	04.07.2004
Staatsangehörigkeit	Pakistan
E-Mail	hammadriaz458@gmail.com
Bewerbungsnummer	BW762641
Abschluss	Master of Science
Studienfach	Artificial Intelligence and Machine Learning, Fachsemester: 4
HZB	Allgemeine Hochschulreife (Ausland)
HZB-Note	2,0
HZB Datum	22.09.2020
Verfahren	frei
Vorstudium	Ja

Obiger Barcode enthält Ihre Bewerbernummer und dient ausschließlich zur Bearbeitung Ihres Antrags.



Studienverlauf

Studienverlauf	Ausland
Angestrebter Abschluss	Abschlussprüfung im Ausland (Erststudium)
Studienfach	Informatik
Name der Hochschule	The Islamia University of Bahawalpur
Von - Bis	21.10.2021 - 30.06.2025
Vollzeit/Teilzeit	
Land	Pakistan
Art Auslandsaufenthalt	Studium

Explanation:

Below is a list of the core courses from TU Darmstadt's Bachelor in Computer Science, along with brief descriptions of each course. To complete this section, identify courses from your previous studies that closely match these topics. Please fill in the right column with the course names and credits from your transcripts that cover similar material. If one of your courses covers topics from more than one of our courses, list it in all relevant fields.

Additionally, be sure to indicate the total credits required for your degree and the standard duration of your program.

Please **do not** submit course description handbooks or comparable documents.

Total credits of degree: _____

Standard duration of the program: _____

Core courses in the Bachelor's degree „Computer Science“ at TU Darmstadt	Successfully passed courses with equivalent topics
Functional and Object-oriented Programming Concepts: • Basic concepts of programming languages • Foundations of functional programming languages • Foundations of object-oriented programming languages • Design and implementation of small software systems • Basic type systems • Fundamental data structures and algorithms and their complexity • Recursion • Simple I/O • Basics of testing • Documenting source code	
Algorithms and Data Structures: • data structures: array, list, binary search tree, b-tree, graph representation, hash table, heaps • algorithms: sorting algorithms, string matching, graph traversal, insertion, search, and deletion on particular data structures, shortest path search, minimal spanning trees • asymptotic complexity • NP completeness • algorithmic strategies: Divide-and-Conquer, dynamic programming, brute-force, greedy, backtracking, meta heuristics	
Digital Design: • Digital Design: digital abstraction and its technological realization, number systems, logic gates, MOSFET transistors and CMOS gates, power consumption • Combinational Logic Design: boolean equations and algebra, mapping equations to gates, multilevel logic circuits, four-valued logic (0,1,X,Z), logic	

minimization, combinational building blocks, timing • Sequential Logic Design: latches, flip-flops, synchronous logic design, finite-state machines, timing, parallelism • Hardware Description Languages: modeling of combinational and sequential circuits, structural modeling, modeling of finite-state machines, data types, parametrized modules, testbenches • Digital Building Blocks: arithmetic circuits, fixed/floating-point representations, sequential building blocks, memory arrays, logic arrays	
Computer Organisation: • Architecture of Microprocessors: programming in assembly and machine language, addressing modes, tool flows, run-time environment • Microarchitecture: instruction set and architectural state, performance analysis, microarchitectures with single-cycle/multi-cycle/pipelined execution, exception handling, advanced microarchitectures • Memory and I/O-Systems: performance analysis, caches, virtual memory, I/O techniques, standard interfaces	
Parallel Programming: • programming languages for systems programming • foundations of parallel systems • parallel architectures, multi-core and many-core systems, clusters • programming paradigms and models for parallel computing • parallel algorithms • significant practical programming exercises covering the above topics	
Operating Systems: • Introduction to Operating Systems (OS) - Role, purpose and design issues • Processes and Threads - OS structures, process control, abstractions, kernel/user modes and operations, context switching, interrupts • Inter-Process Communication - Message passing IPC, RPC, layers, interfaces, hierarchies • Coordination: Deadlocks - Process coordination, critical sections, deadlock characterization, deadlock detection and recovery, deadlock avoidance • Scheduling/Resource Management - Task ordering, preemptive and non-preemptive scheduling, schedulers and policies, OS implementations • Concurrency: Races, Mutual Exclusions - Critical sections, races, spin locks, synchronization • Programming Abstractions: Semaphores - Semaphores, Monitors • Memory Management - Storage structures, management/replacements approaches, virtual memory, paging, caching, segmentation • I/O - Device management, drivers, segmentation,	

interrupt handling, DMA • File systems - File systems requirements, design and implementation, file structures, directories, naming, partitions, virtual file systems • Fault Tolerance/Resilience - Fault types, fault handling approaches, reliable message delivery, OS reliability and availability, security issues • Embedded/RT OS - Memory/disk/performance management, recovery, fault-tolerances, real-time aspects • Distributed OS - Distributed computation and communication abstractions, synchronization, coordination, consistency • Virtual Machines - Purpose and types of virtualization, virtual file systems, Hypervisors	
Introduction to Compiler Construction: • Structure of compilers • Context-free grammars for the description of language syntax • Lexing and parsing techniques • Intermediate representations • Semantic analysis • Run-time organisation • Code generation • Software tools for compiler constructions • Implementation techniques for compilers	
Automata, Formal Languages and Decidability: • introduction: transition systems, words, languages • basic mathematical methods and proof patterns • finite automata and regular languages, determinism and nondeterminism, closure properties and automata constructions, Kleene Theorem, Myhill-Nerode Theorem, pumping lemma • grammars and the Chomsky hierarchy • context-free languages, pumping lemma, CYK algorithm • models of computation: PDA and Turing machines • decidability and recursive enumerability in the Chomsky hierarchy	
Propositional Logic and Predicate Logic: • syntax and semantics of propositional logic, functional completeness and normal forms, compactness, complete proof calculi: resolution and a sequent calculus • syntax and semantics of first-order logic, structures and assignments, normal forms, Skolemization, Herbrand theorem, compactness, complete proof calculi: (ground) resolution and a sequent calculus, Gödel's Completeness Theorem • undecidability of first-order logic • optional: digressions on expressiveness and model Checking	

Formal Methods in Software Design: • Modelling of concurrent software with the ProMeLa language • Formalization of safety and liveness properties in propositional temporal logic • Theoretical Foundations of Model Checking • Verification of ProMeLa programs using the model checker SPIN • Syntax, semantics, and sequent calculus for typed first-order logic • Foundations of the contract-based software specification language JML • Dynamic logic as a first-order program logic • Formal software verification by symbolic execution and invariant reasoning • Tool-based verification of Java programs with the verification system KeY	
Probabilistic Methods in Computer Science: • basics from probability theory, statistics and information theory. • probabilistic approaches to graph-based modeling in computer science • basic probabilistic problems and use of probabilistic methods - o in practical computer science (e.g. run-time analysis of programs, data compression), - o in technical computer science (e.g., reliability of hardware, caching), and - o in applied computer science (e.g., simulation of stochastic systems, probabilistic robotics). • selected randomized algorithms, their analysis by 'The Probabilistic Method', algorithms for automated decision making and optimization • application of probabilistic methods in artificial intelligence (e.g. learning methods, neural networks) and data science • implementation of probabilistic methods by means of practical programming examples	
Introduction to Artificial Intelligence: • Foundations • Introduction, History of AI • Intelligent Agents • Search • Uninformed Search • Heuristic Search • Local Search • Constraint Satisfaction Problems • Games: Adversarial Search • Planning • Planning in State Space • Planning in Plan Space • Decisions under Uncertainty • Uncertainty and Probabilities • Bayesian Networks	

• Decision Making • Machine Learning • Neural Networks • Reinforcement Learning • Philosophical Foundations	
Computer Networks and Distributed Systems: • Foundations: Service, protocols, connection, layer model • protocol mechanisms for media access, routing, broad-/multicast • Multimedia Data Handling • Aspects of continuous data streams and their processing • Quality of service: definition and mechanisms • Multimedia - Synchronization: Basics • Compression procedures	
Computer Security: Part I: Cryptography • Background in Mathematics for cryptography • Security objectives: Confidentiality, Integrity, Authenticity • Symmetric and Asymmetric Cryptography • Hash functions and digital signatures • Protocols for key distribution Part II: IT-Security and Dependability • Basic concepts of IT security • Authentication and biometrics • Access control models and mechanisms • Basic concepts of network security • Basic concepts of software security • Dependable systems: error tolerance, redundancy, Availability	
Information Management: • Information Management Concepts: Information systems concepts Information storage/retrieval, searching, browsing, navigational vs. declarative access • Quality issues: consistency, scalability, availability, reliability • Data Modeling: Conceptual data models (ER/UML structure diagr.), Conceptual design, Operational models (relational model) • Mapping from conceptual to operational model • Relational Model: Operators, Relational algebra, Relational calculus, Implications on query languages derived from RA and RC, Design theory, normalization • Query Languages: SQL (in detail), QBE, Xpath, rdf (high level) • Storage media: RAID, SSDs, Buffering, caching • Implementation of relational operators: Implementation algorithms, Cost functions • Query optimization:	

Heuristic query optimization Cost based query optimization • Transaction processing (concurrency control and recovery): Flat transactions • Concurrency control, correctness criteria: serializability, recoverability, ACA, strictness - Isolation levels, Lock-based schedulers, 2PL - Multiversion concurrency control, Optimistic schedulers, Logging, Checkpointing, Recovery/restart • New trends in data management: Main memory databases, Column stores, NoSQL	
Software Engineering: • Software Project Management • Software Process Models • Requirements Engineering • Software Development Tools • Software Quality; in particular: • Test Processes (automated testing, test coverage metrics, debugging) • Software Metrics • Object-oriented Analysis and Design • Modeling using UML • Software Design Patterns	
Scientific Computing: • Fundamentals of scientific modeling and "The Scientific Method". • Modeling and system description using the example of mechanical systems • Problem specification for the simulation of complex models • Model building and identification using the example of mechanical systems • Model analysis of static systems by numerical methods for the solution of linear and nonlinear systems of equations • Model analysis and simulation of dynamic models by initial value problems with ordinary differential equations • Implementation of models and simulations using examples e.g. from robotics and other fields • Validation of models and simulations using measured data • Applications in the simulation and control of robots as well as physics-based animation and computer games	
Modeling, Specification and Semantics: • introduction to modeling using predicate logic and algebraic concepts • interpretation and faithfulness of formal models • systematic construction of models and making of design decisions • abstraction, refinement, composition, and decomposition of models • syntax and operational semantics of programming languages • elementary proof techniques and their use	

• introduction to specification languages • syntax and denotational semantics of specification languages • modeling communication and coordination in concurrent systems • taxonomy of system properties	
Visual Computing: • Basics of perception • Basic Fourier transformation • Images, filtering, compression & processing • Basic object recognition • Geometric transformations • Basic 3D reconstruction • Surface and scene representations • Rendering algorithms • Color: Perception, spaces & models • Basic visualization	